

Brute Force Sorting Algorithms

Selection Sort, Bubble Sort, Insertion Sort

Laboratory Record

Name: **J Karthik**

Register Number: **192472136**

1. Selection Sort

Q1. Trophy Ranking System

Aim

To display the top k scores of a leaderboard out of 50 participants using selection sort's repeated-maximum approach, without fully sorting the array.

Algorithm

1. Start.
2. Read the array of scores and the required count k.
3. Set $k = \min(k, \text{number of scores})$.
4. For $i = 0$ to $k-1$: scan the unprocessed portion (index i to end) to find the index of the maximum value.
5. Swap the maximum element into position i .
6. After k iterations, return the first k elements as the top-k list.
7. Stop.

Python Program

```
def top_k_scores(scores, k):
    arr = scores.copy()
    n = len(arr)
    k = min(k, n)
    for i in range(k):
        max_idx = i
        for j in range(i + 1, n):
            if arr[j] > arr[max_idx]:
                max_idx = j
        arr[i], arr[max_idx] = arr[max_idx], arr[i]
    return arr[:k]

print(top_k_scores([72, 88, 65, 90, 77, 95, 60, 83, 91, 68], 5))
print(top_k_scores([5, 3, 1], 5))
print(top_k_scores([], 3))
```

Input

```
scores = [72,88,65,90,77,95,60,83,91,68], k = 5
scores = [5,3,1], k = 5
scores = [], k = 3
```

Output

```
[95, 91, 90, 88, 83]
[5, 3, 1]
[]
```

Result

Only k passes are made instead of sorting the entire array of 50 participants, giving $O(k \times n)$ time and $O(1)$ extra space - much cheaper than a full $O(n \log n)$ sort when only a handful of top scores are needed.

Q2. Memory-Constrained Embedded Device

Aim

To sort temperature readings on a low-memory IoT device using selection sort while minimizing the number of memory writes (swaps).

Algorithm

8. Start.
9. Read the array of temperature readings.
10. For $i = 0$ to $n-2$: find the index of the minimum element in the unsorted remainder (i to $n-1$).
11. If the minimum index is not already i , swap and increment the swap counter.
12. If it is already in place, skip the swap (no write performed).
13. Return the sorted array and the total swap count; verify swaps $\leq n-1$.
14. Stop.

Python Program

```
def selection_sort_min_writes(arr):
    a = arr.copy()
    n = len(a)
    swaps = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            a[i], a[min_idx] = a[min_idx], a[i]
            swaps += 1
    return a, swaps

res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
print(res, sw)
res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
print(res2, sw2)
```

Input

```
readings = [23.5, 19.2, 25.1, 18.8, 21.4]
readings2 = [1, 2, 3, 4, 5]
```

Output

```
[18.8, 19.2, 21.4, 23.5, 25.1] swaps = 2
[1, 2, 3, 4, 5] swaps = 0
```

Result

Selection sort performs at most $n-1$ swaps regardless of input order (2 swaps here, 0 for an already sorted list), confirming it minimizes costly memory writes even though comparisons still cost $O(n^2)$ time.

Q3. Library Book Reordering

Aim

To rearrange library books on a shelf by ID number using selection sort, minimizing the number of physical book moves (swaps).

Algorithm

15. Start.
16. Represent each book by its ID in an array.
17. For $i = 0$ to $n-2$: scan the remainder to find the index of the minimum ID (comparisons / label checks, unrestricted).
18. If the minimum is not already at position i , physically swap it into place and increment the move counter.
19. Repeat until the shelf is fully ordered.
20. Return the ordered shelf and total physical moves.
21. Stop.

Python Program

```
def reorder_shelf(ids):
    a = ids.copy()
    n = len(a)
    moves = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            a[i], a[min_idx] = a[min_idx], a[i]
            moves += 1
    return a, moves

ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
print(ordered, moves)
ordered2, moves2 = reorder_shelf([100, 200, 300])
print(ordered2, moves2)
```

Input

```
ids = [305, 102, 250, 118, 199, 400, 101]
ids2 = [100, 200, 300]
```

Output

```
[101, 102, 118, 199, 250, 305, 400] moves = 4
[100, 200, 300] moves = 0
```

Result

Comparisons dominate the runtime ($O(n^2)$ label checks, e.g. $n(n-1)/2 = 190$ for a 20-book shelf), while each shelf position needs at most one physical move, so moves $\leq n-1$ (4 moves here) - confirming selection sort suits scenarios where the swap operation is expensive.

Q4. Contest Prize Distribution

Aim

To repeatedly select the participant with the highest current score and rank all participants in descending order using selection sort logic, without a separate sorting library.

Algorithm

22. Start.
23. Read the list of (name, score) participant tuples.
24. For $i = 0$ to $n-1$: scan the remaining unranked participants to find the one with the highest score.
25. Swap that participant into position i .
26. Print/record the participant as the next-ranked winner.
27. Repeat until all participants are ranked.
28. Stop.

Python Program

```
def distribute_prizes(participants):
    a = participants.copy()
    n = len(a)
    for i in range(n):
        max_idx = i
        for j in range(i + 1, n):
            if a[j][1] > a[max_idx][1]:
                max_idx = j
        a[i], a[max_idx] = a[max_idx], a[i]
    return a

ranking = distribute_prizes([('Asha', 88), ('Ravi', 95), ('Meera', 79), ('Dev', 95)])
print(ranking)
```

Input

```
participants = [('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)]
```

Output

```
[('Ravi', 95), ('Dev', 95), ('Asha', 88), ('Meera', 79)]
```

Result

Each of the n ranking positions requires a full scan of the remaining participants, giving $O(n^2)$ time overall and $O(1)$ extra space, producing the descending ranking (Ravi and Dev tied at 95, ranked ahead of Asha and Meera) without any external sort routine.

Q5. Small Dataset in a Mobile App

Aim

To sort a small 'recently viewed' list of 8-10 items by price using selection sort and to justify its suitability at this scale.

Algorithm

29. Start.
30. Read the array of item prices ($n \leq 10$).
31. For $i = 0$ to $n-2$: find the index of the minimum price in the unsorted remainder.
32. Swap the minimum into position i .
33. Repeat until the array is fully sorted.
34. Measure and display the execution time.
35. Stop.

Python Program

```
import time

def selection_sort(arr):
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        a[i], a[min_idx] = a[min_idx], a[i]
    return a

prices = [499, 129, 899, 45, 275, 60, 310, 150]
start = time.time()
result = selection_sort(prices)
elapsed = time.time() - start
print(result, f"Time: {elapsed * 1e6:.2f} microseconds")
```

Input

prices = [499, 129, 899, 45, 275, 60, 310, 150]

Output

[45, 60, 129, 150, 275, 310, 499, 899] (execution time in the microsecond range)

Result

For such small n , the $O(n^2)$ cost of selection sort is negligible in absolute terms and its simplicity and low swap count make it a reasonable, easy-to-maintain choice, rather than pulling in a general-purpose sort routine for only 8-10 items.

2. Bubble Sort

Q6. Exam Result Slip Correction

Aim

To fix a nearly sorted list of student roll numbers using optimized bubble sort with an early-exit flag, and to show that very few passes are needed.

Algorithm

36. Start.
37. Read the array of roll numbers.
38. Repeat a pass over the array, comparing each pair of adjacent elements.
39. Swap adjacent elements if they are out of order and set a 'swapped' flag to True.
40. Increment the pass counter after each full pass.
41. If a pass completes with no swaps, the array is sorted - exit early.
42. Return the sorted array and the total number of passes used.
43. Stop.

Python Program

```
def optimized_bubble_sort(arr):  
    a = arr.copy()  
    n = len(a)  
    passes = 0  
    for i in range(n):  
        passes += 1  
        swapped = False  
        for j in range(n - i - 1):  
            if a[j] > a[j + 1]:  
                a[j], a[j + 1] = a[j + 1], a[j]  
                swapped = True  
        if not swapped:  
            break  
    return a, passes
```

```
sorted_rolls, passes = optimized_bubble_sort([101, 102, 104, 103, 105, 107, 106, 108])  
print(sorted_rolls, passes)  
sorted_ok, passes_ok = optimized_bubble_sort([1, 2, 3, 4, 5])  
print(sorted_ok, passes_ok)
```

Input

rolls = [101, 102, 104, 103, 105, 107, 106, 108] (2-3 out of place)
rolls2 = [1, 2, 3, 4, 5] (already sorted)

Output

```
[101, 102, 103, 104, 105, 106, 107, 108] passes = 2  
[1, 2, 3, 4, 5] passes = 1
```

Result

Only 2 passes were needed (far fewer than the full $n=8$ passes) because just two roll numbers were out of place, and the already-sorted list exits after a single pass - confirming the $O(n)$ best-case behaviour of optimized bubble sort on nearly sorted data.

Q7. Traffic Signal Priority Queue

Aim

To reorder a small queue of vehicles by priority (ambulance > bus > car) using bubble sort each time a new vehicle arrives, and to discuss the trade-off between simplicity and efficiency.

Algorithm

44. Start.
45. Assign a numeric priority to each vehicle type (ambulance=1, bus=2, car=3).
46. Read the current queue and append the newly arrived vehicle.
47. Repeat passes over the queue, comparing adjacent vehicles' priorities.
48. Swap adjacent vehicles if the left one has a lower priority (higher number) than the right one.
49. Continue until no swaps are needed in a pass.
50. Return the reordered queue.
51. Stop.

Python Program

```
def bubble_sort_queue(queue):
    priority = {'ambulance': 1, 'bus': 2, 'car': 3}
    a = queue.copy()
    n = len(a)
    for i in range(n):
        for j in range(n - i - 1):
            if priority[a[j]] > priority[a[j + 1]]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a

queue = ['car', 'car', 'bus']
queue.append('ambulance')
result = bubble_sort_queue(queue)
print(result)
print(bubble_sort_queue(['ambulance']))
```

Input

```
queue = ['car', 'car', 'bus', 'ambulance']
queue2 = ['ambulance']
```

Output

```
['ambulance', 'bus', 'car', 'car']
['ambulance']
```

Result

Re-sorting the whole small queue on every arrival costs $O(n^2)$, which is acceptable since n stays very small at an intersection; bubble sort's simplicity makes it easy to verify for correctness in this safety-critical setting, outweighing its inefficiency at this scale.

Q8. Bubble Sort Visualization Tool

Aim

To implement bubble sort so that it records the state of the array after every pass, for use in an educational animation showing how the largest elements 'bubble' to the top.

Algorithm

52. Start.
53. Read the array and store its initial state as frame 0.
54. For each outer-loop pass, compare adjacent elements and swap if out of order.
55. After completing each pass, append the current array state as a new frame.
56. Repeat for all n passes.
57. Return the full list of frames for animation.
58. Stop.

Python Program

```
def bubble_sort_with_frames(arr):
    a = arr.copy()
    n = len(a)
    frames = [a.copy()]
    for i in range(n):
        for j in range(n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
        frames.append(a.copy())
    return frames

frames = bubble_sort_with_frames([5, 1, 4, 2, 8])
for idx, f in enumerate(frames):
    print(f"Frame {idx}: {f}")
```

Input

```
arr = [5, 1, 4, 2, 8]
```

Output

```
Frame 0: [5, 1, 4, 2, 8]
Frame 1: [1, 4, 2, 5, 8]
Frame 2: [1, 2, 4, 5, 8]
Frame 3: [1, 2, 4, 5, 8]
Frame 4: [1, 2, 4, 5, 8]
Frame 5: [1, 2, 4, 5, 8]
```

Result

The final frame equals the fully sorted array; storing one frame per pass costs $O(n)$ work each, giving $O(n^2)$ space in the worst case across $n+1$ frames - well suited to driving a step-through animation.

Q9. Sorting Sensor Alerts by Severity

Aim

To sort 15 monitoring alerts by severity level using both plain and optimized bubble sort, and to measure whether early-termination meaningfully reduces comparisons.

Algorithm

59. Start.
60. Read the array of severity levels.
61. Plain bubble sort: always perform the full n-1 passes, counting every comparison made.
62. Optimized bubble sort: use a 'swapped' flag to exit as soon as a pass makes no swaps, counting comparisons.
63. Sort the same data with both versions.
64. Compare the two comparison counts.
65. Stop.

Python Program

```
def bubble_sort_plain(arr):
    a = arr.copy()
    n = len(a)
    comparisons = 0
    for i in range(n - 1):
        for j in range(n - i - 1):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a, comparisons

def bubble_sort_optimized(arr):
    a = arr.copy()
    n = len(a)
    comparisons = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a, comparisons

alerts = [2, 1, 3, 2, 1, 4, 3, 2, 5, 1, 2, 3, 4, 1, 2]
r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)
print(r1 == r2)
print('Plain comparisons:', c1, '| Optimized comparisons:', c2)
```

Input

alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2] (n = 15)

Output

Both produce the same sorted result:

[1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 4, 4, 5]

Plain comparisons: 105 | Optimized comparisons: 99

Result

The plain version always performs $14+13+\dots+1 = 105$ comparisons. The optimized version needs only 99, a modest saving here because the data is not close to fully sorted; the benefit of early-exit grows much larger as the input approaches sorted order.

Q10. Card Game Hand Sorting

Aim

To sort a 13-card hand after a single new card is drawn using optimized bubble sort, and to compare the pass count to a full re-sort.

Algorithm

66. Start.
67. Read the current (sorted) hand and append the newly drawn card.
68. Repeat passes comparing adjacent cards, swapping if out of rank order, using an early-exit flag.
69. Count the number of passes required for the nearly-sorted hand.
70. Repeat the same process on a fully shuffled version of the same hand for comparison.
71. Compare the pass counts of the nearly-sorted and fully shuffled cases.
72. Stop.

Python Program

```
def bubble_sort_hand(hand):
    a = hand.copy()
    n = len(a)
    passes = 0
    for i in range(n):
        passes += 1
        swapped = False
        for j in range(n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a, passes

hand = [2, 4, 6, 8, 9, 11, 13]
hand.append(7) # nearly sorted: one new card
final_hand, passes_incremental = bubble_sort_hand(hand)
print(final_hand, passes_incremental)
```

Input

hand = [2, 4, 6, 8, 9, 11, 13], new card = 7 -> [2,4,6,8,9,11,13,7]

Output

[2, 4, 6, 7, 8, 9, 11, 13] passes = 5

Result

Because only the single new card (7) needs to bubble into place, the sort completes in a handful of passes (approx. $O(n)$ behaviour), well below the up-to- n passes a fully shuffled 8-card hand would require, confirming bubble sort's efficiency on nearly sorted data.

3. Insertion Sort

Q11. Live Leaderboard Updates

Aim

To insert a single updated player score into its correct position in an already-sorted leaderboard using insertion sort, instead of re-sorting the entire list.

Algorithm

73. Start.
74. Read the current sorted leaderboard (descending order) and the updated score.
75. Remove the player's old entry and append the new score at the end.
76. Set key = new score and shift leaderboard entries that are smaller than key one position to the right.
77. Insert key into the vacated position.
78. Count and return the number of shifts performed.
79. Stop.

Python Program

```
def insert_updated_score(board, new_score):
    a = board.copy()
    a.append(new_score)
    n = len(a)
    shifts = 0
    key = a[n - 1]
    j = n - 2
    while j >= 0 and a[j] < key:
        a[j + 1] = a[j]
        j -= 1
        shifts += 1
    a[j + 1] = key
    return a, shifts

board = [980, 875, 760, 690, 500]
updated_board, shifts = insert_updated_score(board, 820)
print(updated_board, shifts)
board2, shifts2 = insert_updated_score(board, 100)
print(board2, shifts2)
```

Input

board = [980, 875, 760, 690, 500], new score = 820
board = [980, 875, 760, 690, 500], new score = 100

Output

[980, 875, 820, 760, 690, 500] shifts = 2
[980, 875, 760, 690, 500, 100] shifts = 0

Result

Only the shifting needed to place the single new score is performed (2 shifts for 820, 0 shifts for the lowest score 100), giving $O(n)$ time in place of a full $O(n \log n)$ re-sort, with $O(1)$ extra space via in-place shifting.

Q12. Card Sorting While Playing

Aim

To sort a growing hand of playing cards, card by card, exactly as a player would while picking up cards one at a time - the classic real-world analogy for insertion sort.

Algorithm

80. Start.
81. Initialize an empty hand.
82. For each newly picked-up card: append it to the hand.
83. Set key = new card and j = second-last index.
84. While j $\geq$ 0 and hand[j] > key: shift hand[j] to hand[j+1] and decrement j.
85. Insert key at position j+1.
86. Repeat for every card picked up.
87. Stop.

Python Program

```
def pick_up_card(hand, card):
    a = hand.copy()
    a.append(card)
    j = len(a) - 2
    key = a[-1]
    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
    return a

hand = []
for card in [7, 2, 9, 4, 1]:
    hand = pick_up_card(hand, card)
print(hand)
print(pick_up_card([], 5))
```

Input

cards picked up in order: 7, 2, 9, 4, 1

Output

[1, 2, 4, 7, 9]
[5] (first card picked up)

Result

Each new card requires shifting past several existing cards, giving $O(n)$ work per card and $O(n^2)$ total over n cards, with $O(1)$ extra space since cards are shifted in place within the hand list.

Q13. Real-Time Stock Price Feed

Aim

To maintain a continuously sorted list of stock prices as new prices arrive one at a time, using insertion sort's incremental insertion technique for fast min/max lookup.

Algorithm

88. Start.
89. Initialize an empty sorted price list.
90. For each incoming price: append it to the list.
91. Set key = new price and shift larger prices one position to the right (linear scan from the right).
92. Insert key into the vacated position.
93. Repeat for every incoming price; min and max are now $O(1)$ lookups (first and last elements).
94. Stop.

Python Program

```
def insert_price(prices, p):
    a = prices.copy()
    a.append(p)
    j = len(a) - 2
    key = a[-1]
    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
    return a

prices = []
for p in [102.5, 98.3, 105.1, 100.0, 97.8]:
    prices = insert_price(prices, p)
print(prices)
print(prices[0] == min(prices) and prices[-1] == max(prices))
```

Input

incoming prices in order: 102.5, 98.3, 105.1, 100.0, 97.8

Output

[97.8, 98.3, 100.0, 102.5, 105.1]
True ($O(1)$ min/max lookup confirmed)

Result

Finding the insertion point is fast, but shifting elements to make room is still $O(n)$ per insertion, giving $O(n^2)$ worst case overall for n prices - still far better than re-sorting the whole list from scratch on every arrival.

Q14. Playlist Reordering by Recently Added

Aim

To insert a newly added song into a playlist that is already sorted by duration, placing just the new song correctly instead of sorting the whole playlist again.

Algorithm

95. Start.
96. Read the sorted playlist (list of (name, duration) tuples) and the new song.
97. Append the new song's duration to the playlist.
98. Set key = new song and shift longer songs one position to the right.
99. Insert key into the vacated position.
100. Return the updated playlist.
101. Stop.

Python Program

```
def insert_song(playlist, song):  
    a = playlist.copy()  
    a.append(song)  
    j = len(a) - 2  
    key = a[-1]  
    while j >= 0 and a[j][1] > key[1]:  
        a[j + 1] = a[j]  
        j -= 1  
    a[j + 1] = key  
    return a
```

```
playlist = [('Intro', 120), ('Chill Beat', 210), ('Long Jam', 340)]  
updated_playlist = insert_song(playlist, ('Quick Track', 180))  
print(updated_playlist)
```

Input

playlist = [('Intro',120), ('Chill Beat',210), ('Long Jam',340)], new song = ('Quick Track', 180)

Output

[('Intro', 120), ('Quick Track', 180), ('Chill Beat', 210), ('Long Jam', 340)]

Result

Only the new song ('Quick Track', 180 seconds) is shifted into place among the already-sorted songs, giving $O(n)$ time and $O(1)$ extra space via in-place shifting within the playlist list.

Q15. Nearly Sorted Sensor Log Correction

Aim

To fix a mostly-chronological weather sensor log using insertion sort, and to show that the number of shifts required is very low compared to a randomly shuffled version of the same data.

Algorithm

102. Start.
103. Read the temperature log array.
104. For $i = 1$ to $n-1$: set $key = \log[i]$ and $j = i-1$.
105. While $j \geq 0$ and $\log[j] > key$: shift $\log[j]$ to $\log[j+1]$, decrement j , and increment the shift counter.
106. Insert key at position $j+1$.
107. Repeat for the entire log, then repeat the same process on a randomly shuffled copy for comparison.
108. Compare the shift counts of the nearly-sorted and shuffled versions.
109. Stop.

Python Program

```
import random

def insertion_sort_count_shifts(arr):
    a = arr.copy()
    shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
            shifts += 1
        a[j + 1] = key
    return a, shifts

log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]
sorted_log, shifts_nearly = insertion_sort_count_shifts(log)
print(sorted_log, shifts_nearly)

shuffled_log = log.copy()
random.shuffle(shuffled_log)
_, shifts_random = insertion_sort_count_shifts(shuffled_log)
print('Nearly-sorted shifts:', shifts_nearly, '| Random shifts (this run):', shifts_random)
```

Input

log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0] (only entry 17.9 delayed / out of place)

Output

Sorted log: [17.9, 18.2, 18.5, 18.9, 19.0, 19.1, 19.4]
Nearly-sorted shifts: 5 | Random shifts: typically much higher (varies by shuffle)

Result

Because only a few entries are out of place, most elements require zero or one shift, giving a shift count close to the best-case $\Omega(n)$; a randomly shuffled version needs close to the worst-case $O(n^2)$ shifts, confirming insertion sort's sensitivity to input order.